Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИип2

Гаревская Я.А.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 4
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 6
3.3. Описание алгоритмов 7
4. Результаты экспериментов и тестирования 9
Заключение 10
Литература 11
Приложение 12

Введение

Множество является одной из базовых структур дискретной математики. Оно представляет собой совокупность различных элементов, для которых определены операции включения, удаления, проверки принадлежности, объединения, пересечения и дополнения.

Для хранения конечного множества удобно использовать битовое поле: каждому элементу универсального множества соответствует один бит. Если бит равен 1, элемент входит в множество; если бит равен 0, элемент отсутствует. Такой способ позволяет компактно хранить данные и выполнять основные операции с помощью побитовых инструкций.

Цель лабораторной работы - реализовать классы TBitField и TSet на языке C++, изучить управление динамической памятью, перегрузку операторов и проверить работу разработанных классов с помощью Google Test.

1. Постановка задачи

Требуется реализовать класс TBitField, который хранит битовое поле заданной длины в динамическом массиве элементов типа TELEM. Класс обеспечивает создание и копирование поля, освобождение памяти, получение длины, установку, сброс и чтение отдельного бита, сравнение, присваивание, побитовые операции ИЛИ, И и НЕ, а также потоковый ввод и вывод.

На основе TBitField реализуется класс TSet, представляющий конечное множество неотрицательных целых чисел. Для множества доступны добавление и удаление элемента, проверка принадлежности, сравнение, присваивание, объединение, пересечение, дополнение, операции добавления и удаления элемента с созданием нового множества, преобразование к TBitField и потоковый ввод/вывод.

В предоставленных заголовочных файлах tbitfield.h и tset.h заданы интерфейсы классов. Реализация методов размещена в файлах tbitfield.cpp и tset.cpp. Для проверки используются автоматические тесты Google Test, а файл main.cpp демонстрирует создание множества и вывод его содержимого.

В данной версии программы границы множества реализованы включительно: объект TSet(mp) содержит битовое поле длиной mp + 1, поэтому допустимые элементы имеют номера от 0 до mp. Методы доступа при выходе за границы не выбрасывают исключение, а игнорируют некорректный индекс. Побитовые операции над TBitField требуют одинаковой длины операндов. Эти особенности отражены в описании фактической реализации и тестов.

2. Руководство пользователя

Программа состоит из библиотечных классов и демонстрационного приложения. Для запуска демонстрации необходимо собрать файлы main.cpp, tset.cpp и tbitfield.cpp вместе. Для запуска автоматических тестов дополнительно подключается Google Test и файл test.cpp.

В демонстрационной программе создаётся множество TSet setA(10), после чего в него добавляются элементы 1, 3 и 5. Затем оператор вывода отображает множество в фигурных скобках. Ожидаемый результат работы main.cpp:

```
Set A: { 1, 3, 5 }
```

При использовании класса TSet параметр конструктора задаёт максимальный номер элемента универсума включительно. Например, TSet(3) допускает элементы 0, 1, 2 и 3. Для добавления элемента используется InsElem(e), для удаления — DelElem(e), для проверки принадлежности — IsMember(e).

Операторы + и * между двумя объектами TSet выполняют объединение и пересечение соответственно. Оператор ~ формирует дополнение относительно универсума данного объекта. Операторы + e и - e возвращают копию множества с добавленным или удалённым элементом, не изменяя исходный объект.

Для ввода множества оператор >> считывает целые числа до появления значения -1. Все элементы перед вводом очищаются; числа, не входящие в диапазон множества, не добавляются. Для вывода оператор << формирует строку вида { 1, 3, 5 }. Для TBitField вводится последовательность символов для каждой позиции поля; выводится последовательность битов от нулевого индекса к последнему.

3. Руководство программиста

3.1. Описание структуры программы

Проект состоит из следующих файлов:

tbitfield.h — заголовочный файл с объявлением класса TBitField, типом TELEM и прототипами методов.

tbitfield.cpp — реализация методов класса TBitField и операторов потокового ввода/вывода.

tset.h — заголовочный файл с объявлением класса TSet и его интерфейса.

tset.cpp — реализация методов класса TSet и операторов потокового ввода/вывода.

main.cpp — демонстрационная программа, создающая множество, добавляющая элементы и выводящая результат.

test.cpp — набор модульных тестов Google Test для проверки классов TBitField и TSet.

Заголовочные файлы предоставлены преподавателем. В них объявлены поля классов, конструкторы, деструкторы, методы доступа и перегруженные операторы. Файлы .cpp содержат определения этих методов.

3.2. Описание структур данных

Класс TBitField

Класс TBitField реализует битовое поле на основе динамического массива машинных слов. Тип TELEM определён как unsigned int. В реализации размер одного слова в битах вычисляется как sizeof(TELEM) * 8.

BitLen — количество битов в поле.

pMem — указатель на динамический массив элементов TELEM, в котором хранятся значения битов.

MemLen — количество элементов TELEM, выделенных для хранения поля.

Номер слова для бита n вычисляется целочисленным делением n на количество битов в TELEM. Маска бита формируется сдвигом единицы на n % (sizeof(TELEM) * 8) позиций. Конструктор выделяет память и обнуляет все слова. Конструктор копирования создаёт отдельный массив и копирует содержимое. Деструктор освобождает массив оператором delete[].

Класс TSet

Класс TSet реализует конечное множество с помощью объекта TBitField.

MaxPower — максимальный допустимый номер элемента множества.

BitField — битовое поле, в котором бит с индексом i показывает наличие элемента i в множестве.

Конструктор TSet(int mp) создаёт битовое поле длиной mp + 1. Поэтому в данной реализации универсум включает элементы от 0 до MaxPower включительно. Конструктор преобразования из TBitField устанавливает MaxPower равным длине поля минус один. Оператор преобразования возвращает хранящееся битовое поле.

Глобальные данные

Глобальные переменные и пользовательские структуры данных в проекте не используются. Все данные, необходимые для работы, хранятся в полях объектов TBitField и TSet.

3.3. Описание алгоритмов

1. Адресация бита в массиве

Для обращения к биту n вычисляется индекс машинного слова GetMemIndex(n) = n / B, где B = sizeof(TELEM) * 8. Маска GetMemMask(n) равна 1u << (n % B). Обе операции выполняются за O(1).

2. Установка, сброс и чтение бита

Перед обращением проверяется условие $0 \le n < \text{BitLen}$. Если индекс находится вне диапазона, SetBit и ClrBit ничего не изменяют, а GetBit возвращает 0. Для установки применяется побитовое ИЛИ: pMem[index] |= mask. Для сброса используется побитовое И с инверсной маской: pMem[index] &= ~mask. Чтение выполняется проверкой pMem[index] & mask. Каждая операция имеет сложность O(1).

3. Копирование и присваивание

Конструктор копирования переносит значения BitLen и MemLen, выделяет новый массив pMem и копирует в него каждое машинное слово. Оператор присваивания сначала проверяет самоприсваивание. Если размеры массивов различаются, старый массив освобождается и выделяется новый. Затем копируются длина поля и содержимое массива. Сложность копирования — O(MemLen).

4. Сравнение битовых полей

Операторы == и != сначала сравнивают длины полей. При разных длинах поля считаются неравными. При одинаковой длине последовательно сравниваются элементы массива pMem. Если все слова совпадают, поля равны. Сложность — O(MemLen).

5. Побитовые операции

Операторы | и & проверяют совпадение BitLen операндов. Если длины различаются, выбрасывается std::invalid_argument. При одинаковой длине создаётся поле результата той же длины, после чего каждое слово вычисляется побитовой операцией ИЛИ или И. Оператор ~ инвертирует все слова. Если длина не кратна размеру машинного слова, маска обнуляет лишние биты последнего слова, находящиеся за пределами BitLen. Сложность операций — O(MemLen).

6. Операции над множествами

InsElem(e) устанавливает бит e, DelElem(e) сбрасывает его, IsMember(e) возвращает значение соответствующего бита. Все три метода проверяют диапазон $0 \le e \le \text{MaxPower}$; некорректные номера игнорируются или дают результат 0. Операторы + и * для двух множеств проверяют равенство MaxPower и при несовпадении выбрасывают std::invalid_argument. При совпадении выполняют побитовое ИЛИ или И над внутренними полями. Оператор ~ инвертирует битовое поле, формируя дополнение относительно заданного диапазона.

Операторы + e и - e создают копию исходного множества, затем добавляют или удаляют элемент в копии. Исходный объект остаётся без изменений. Сложность операций с одним элементом — O(1), операций над двумя множествами и дополнения — O(MemLen).

7. Потоковый ввод и вывод

Оператор вывода TBitField перебирает индексы от 0 до BitLen - 1 и печатает значение каждого бита. Оператор ввода считывает по одному символу для каждой позиции: символ 1 устанавливает бит, остальные символы очищают его. Оператор вывода TSet перебирает элементы от 0 до MaxPower и печатает индексы установленных битов через запятую внутри фигурных скобок. Оператор ввода TSet предварительно очищает множество, затем считывает целые числа до разделителя -1; каждый считанный элемент добавляется через InsElem.

Для обоих классов основные операции используют прямую адресацию битов, поэтому отдельные операции доступа выполняются за O(1), а операции по всему массиву — за O(MemLen).

4. Результаты экспериментов и тестирования

Для проверки реализации подготовлен файл test.cpp, содержащий 18 тестов Google Test: 9 тестов для TBitField и 9 тестов для TSet. Тесты проверяют штатные сценарии, граничные значения, копирование, присваивание, перегруженные операторы и потоковый ввод/вывод.

Тестирование TBitField

- Initialization — проверяет длину поля и нулевое начальное состояние всех битов.
- EdgeZeroLength — проверяет создание поля нулевой длины.
- SetAndClear — проверяет установку и сброс отдельного бита.
- OutOfBounds — проверяет поведение при обращении к отрицательному и слишком большому индексу; ожидается отсутствие изменения поля и возврат 0 при чтении.
- CopyConstructor — проверяет копирование длины и установленных битов.
- AssignmentOperator — проверяет присваивание поля другой длины.
- Equality — проверяет операторы равенства и неравенства.
- LogicOperations — проверяет побитовые ИЛИ, И и НЕ.
- IOOperator — проверяет чтение и запись последовательности битов.

Тестирование TSet

- InsAndDel — проверяет добавление, проверку принадлежности и удаление элемента.
- EdgeCases — проверяет игнорирование элементов за пределами допустимого диапазона.
- UnionOperation — проверяет объединение двух множеств.
- Intersection — проверяет пересечение двух множеств.
- Complement — проверяет дополнение множества относительно диапазона от 0 до MaxPower включительно.
- AddSubElementOperator — проверяет добавление и удаление элемента с помощью операторов + и -, без изменения исходного объекта.
- EqualityAndAssignment — проверяет сравнение и присваивание множеств.
- ConversionOperators — проверяет преобразование TSet в TBitField и создание TSet из TBitField.
- IOOperator — проверяет ввод множества до значения -1 и вывод в формате фигурных скобок.

В тестах используются контрольные примеры с заранее известным результатом. Например, для объединения множеств {1, 2} и {2, 3} ожидается {1, 2, 3}; для пересечения множеств {1, 2, 3} и {2, 3, 4} ожидается {2, 3}. Проверка ввода и вывода подтверждает получение строки { 1, 3, 5 } для соответствующего набора элементов.

Набор тестов соответствует фактическому поведению представленного кода: индексы вне диапазона не вызывают исключений, а игнорируются; длина поля TSet равна MaxPower + 1; побитовые операции TBitField выполняются только для полей одинаковой длины. Наличие тестов описывает предусмотренные проверки, но само по себе не является подтверждением их успешного запуска в конкретной среде.

Заключение

В ходе лабораторной работы реализованы два взаимосвязанных класса: TBitField для хранения битовых последовательностей и TSet для представления конечных множеств на их основе.

В TBitField реализованы динамическое выделение и освобождение памяти, копирование, присваивание, доступ к отдельным битам, сравнение, побитовые операции и потоковый ввод/вывод. В TSet реализованы добавление и удаление элементов, проверка принадлежности, сравнение, операции объединения, пересечения и дополнения, операции с отдельным элементом, преобразование типов и потоковый ввод/вывод.

Для проверки основных возможностей подготовлен набор из 18 модульных тестов Google Test. Тесты охватывают создание объектов, граничные случаи, работу операторов, копирование и ввод/вывод. Демонстрационная программа показывает создание множества и вывод его элементов.

Таким образом, поставленная задача по реализации битового поля и множества на его основе выполнена в рамках интерфейсов, заданных в заголовочных файлах.

Литература

Страуструп Б. Язык программирования C++. Специальное издание. Пер. с англ. — М.: Издательство Бином, 2017. — 1136 с.

cppreference.com. Справочник по языку C++ и стандартной библиотеке. URL: https://en.cppreference.com/ (дата обращения: 06.11.2026).

GoogleTest. Документация фреймворка Google Test. URL: https://google.github.io/googletest/ (дата обращения: 06.11.2026).

Приложение

В приложении приведён фрагмент демонстрационной программы main.cpp, используемой для проверки создания множества, добавления элементов и вывода результата.

```
#include "tset.h"
#include <iostream>

int main() {
    TSet setA(10);
    setA.InsElem(1);
    setA.InsElem(3);
    setA.InsElem(5);
    std::cout << "Set A: " << setA << std::endl;
    return 0;
}
```

Ожидаемый вывод: Set A: { 1, 3, 5 }.